Maintenance Programming

Final Project Report

Maintenance of the legacy Node.js `request` library

Group 4
Martin Ek • Andreas Lund • Prodip Kumar Das
Course: 1DL601 Maintenance Programming
Date: 21 October 2026

Abstract

This report documents Group 4's maintenance of the deprecated Node.js `request` HTTP client library. The work focused on making the legacy test infrastructure usable in the current Windows environment, investigating and repairing test-certificate problems, improving shell-script maintainability, and addressing specific test and agent-handling failures. Changes included cross-platform test discovery and startup, Windows named-pipe support, certificate fixture updates, certificate serial-number handling, a proxy agent-pool correction, test-server handling for a multipart authentication test, and shell-script line-ending checks. The latest documented Windows validation passed 1,606 assertions, including browser tests. The final branch was also validated on macOS, where linting, all 56 test files (1,606/1,606 assertions), and the browser tests passed. The branch has not been verified on Linux. The final maintenance classification treats HTTPS certificate fixtures, Windows test-infrastructure support, form-data test-server response handling, `localAddress` test isolation, and browser TLS configuration as Adaptive Maintenance; shell-script portability as Perfective Maintenance; and the separate agent-pool protocol collision fix as Corrective Maintenance.

1. Introduction and Legacy Code Description

The project concerns `request`, a legacy HTTP client library for Node.js. The original project was deprecated in February 2020. We selected a fork so that maintenance work could be performed without modifying the original upstream repository.

Source repository: https://github.com/prodip5619/request

The library is written primarily in JavaScript and runs on Node.js. The repository also contains shell scripts, certificate fixtures, test servers, and a legacy browser-test stack based on Karma and PhantomJS. These supporting components are part of the maintenance scope because the project's tests must run reliably to validate changes.

1.1 Project goals

- Establish a reproducible baseline and understand why the existing test suite failed.
- Identify root causes before changing production-library behavior.
- Make small, evidence-based changes while preserving strict TLS certificate validation.
- Improve the reliability and maintainability of the test infrastructure.
- Document test results, platform limits, and each group member's contribution.

2. Initial State and Problem Investigation

During initial setup on Windows, `npm install` failed with an ERESOLVE dependency-tree error involving `karma@3.1.4` and `karma-tap@3.2.1`. We used `npm install --legacy-peer-deps` to install the existing legacy dependency set without automatically upgrading packages. The installation reported deprecated packages and 59 security vulnerabilities. We did not run `npm audit fix`, because broad dependency changes would have made it harder to distinguish existing legacy problems from newly introduced changes.

The initial `npm test` run passed linting but failed in `test-ci`, so the browser tests were not reached in that run. The aggregate output did not clearly identify the failing test, so individual tests were run directly.

2.1 Initial HTTPS certificate failure

The initial `tests/test-https.js` run produced 20 assertions: 10 passed and 10 failed. Relaxed HTTPS checks passed, while strict checks failed with `CERT_HAS_EXPIRED`. Inspection found that the CA certificate had expired on 27 February 2022, while the server certificate was still within its validity period at that point. OpenSSL verification also failed. The investigation did not find evidence that `request.js` incorrectly propagated the CA or `rejectUnauthorized` options. We therefore focused on test fixtures and the test environment rather than weakening TLS verification.

2.2 Other failures investigated

Observed issue	Investigation finding
Windows test discovery	A wildcard was passed through `cmd.exe` to Taper without being expanded, so the intended test-file list was not passed to the runner.
Unix socket test on Windows	The Unix-domain socket path was not suitable for Windows. The Windows path needed a named pipe while retaining Unix-domain sockets on other platforms.
Browser test startup	Karma startup needed a Windows-specific command-resolution path.
Proxy agent pooling	A tunnel regression test exposed incompatible pooled-agent selection when protocol information was keyed incorrectly.
Multipart form-data test	A test-server 401 response could lead to `ECONNRESET` while the client was still sending a multipart request body.

3. Methods and Tools Used

- Git history, branches, commits, and pull-request inspection.
- Direct execution of individual tests to isolate failures hidden by aggregate test-runner output.
- Full-suite, coverage, browser, HTTPS, tunnel, OAuth, Unix/named-pipe, form-data, and shell-script checks.
- OpenSSL inspection of certificate subjects, issuers, validity periods, key sizes, signatures, chains, hostnames, and key/certificate matching.
- Inspection of Karma/PhantomJS configuration and Windows-specific process-launch behavior.
- Recording investigation findings and verification status in `MAINTENANCE.md`.

We distinguished diagnostic experiments from permanent fixes. Lowering OpenSSL security settings was used in an earlier investigation to understand compatibility failures, not as the final solution. The maintenance changes did not intentionally disable certificate verification.

4. Maintenance Activities and Changes

Each activity is assigned one primary maintenance type, even when a change has characteristics associated with more than one type. The classifications below reflect our final decision for this project and distinguish core test-suite restoration work from separate Perfective and Corrective Maintenance activities.

4.1 Adaptive maintenance - HTTPS certificate fixtures and generation

The initial HTTPS investigation found that strict certificate verification failed because the test CA had expired in February 2022. The certificate fixtures were renewed so that the HTTPS tests could run again. This was part of the core restoration work: the existing test suite could not be considered operational while its HTTPS tests failed because of obsolete test certificates.

The HTTPS test key `tests/ssl/test.key` was replaced with a 2048-bit key and a SHA-256 certificate, generated by `tests/ssl/gen-test.sh` (commit `6fbcb56`, PR #3). The previous key was retained as `tests/ssl/oauth-rsa.key` for `tests/test-oauth.js`, whose expected signatures depend on it; it is used for signing, not TLS.

Strict certificate validation was retained. Lowering the OpenSSL security level was used only as a diagnostic experiment and was not part of the final fix. Martin's later macOS validation and certificate-key/serial-number changes remain attributed to Martin in Section 9.

4.2 Adaptive maintenance - Windows test-infrastructure support

The test infrastructure was adapted to Windows behavior while retaining Unix-like paths where applicable. Changes included explicit test-file discovery, Windows named-pipe support, and browser test startup handling. These changes addressed environment-specific barriers encountered while restoring the core test suite.

The goal was to adapt the test infrastructure to the execution environment without changing intended library behavior. Windows tests later passed. This does not establish that the branch passes on every operating system.

4.3 Adaptive maintenance - Form-data test-server response during core restoration

During the original test-suite restoration, `tests/test-form-data.js` failed in the multipart basic-auth case: the server returned a 401 response while the client was still transmitting the multipart body, resulting in `ECONNRESET` and a subsequent missing-response error. The test server was changed to send `Connection: close` with the 401 response. The form-data test subsequently passed 64/64 assertions.

This change is classified as adaptive maintenance within the core restoration work because it adapts the test harness's response handling to the request/test execution behavior that was preventing the existing suite from passing. It is not counted as a separate additional Perfective or Corrective activity.

4.4 Adaptive maintenance - localAddress test isolation

The `localAddress` test was changed to use a local HTTP server instead of an external Google endpoint. This reduces dependence on external network availability and makes the test exercise the intended behavior in a controlled local environment. The change is treated as part of the core test-suite restoration and classified as adaptive maintenance, rather than being counted as a separate additional maintenance activity.

4.5 Adaptive maintenance - browser TLS configuration

The browser tests used separate fixtures under `tests/browser/ssl/`. These fixtures were expired, used 1024-bit RSA keys and SHA-1 signatures, and the Karma configuration ignored SSL errors. Consequently, a passing browser test did not demonstrate normal certificate trust validation.

The browser-only CA and server certificate/key were replaced with a 2048-bit RSA test CA and a matching server certificate signed with SHA-256. `DNS:localhost` was included in the server certificate's Subject Alternative Name, and the chain and hostname were verified.

`tests/browser/karma.conf.js` was updated to provide the test CA to PhantomJS and set `--ignore-ssl-errors=false`. `tests/browser/test.js` was updated so TLS request errors become explicit failed assertions rather than a secondary missing-response exception.

The positive case (trusted CA and correct hostname) and negative cases (unrelated CA and incorrect hostname) were checked. This browser-specific work is classified as adaptive maintenance because it adapts the browser test environment and certificate setup to current TLS validation requirements.

4.6 Perfective maintenance - shell-script portability and maintainability

After the main test suite had become operational, the shell scripts were reviewed for portability and maintainability. Inconsistent CRLF line endings had caused a Bash execution problem in `tests/ssl/ca/gen-all-certs.sh`. The work added an `.gitattributes` LF policy, normalized relevant scripts, documented the LF requirement, and added `tests/test-shell-scripts.js`.

The shell-script test performs 12 checks covering carriage-return characters and `bash -n` syntax validation. This is classified as a separate Perfective activity because it improves consistency and maintainability of supporting scripts without changing the library's runtime behavior. These checks do not execute the certificate-generation scripts or prove compatibility on every platform.

4.7 Corrective maintenance - agent-pool protocol collision (separate activity)

A tunnel regression test exposed a protocol mismatch in pooled-agent selection. The change in `request.js` keys pooled agents using the proxy protocol for non-tunneled proxy requests and otherwise uses the destination protocol. The tunnel test exercises an HTTPS request early enough to expose incompatible pooled-agent selection. A recorded in-memory old-key counterfactual passed 23/27 assertions and produced four protocol errors; the updated tunnel test passed 27/27.

This is classified as a separate Corrective Maintenance activity because it fixes a functional protocol-selection defect in the library, rather than merely adapting the test infrastructure to the environment. The project evidence places this focused fix in the later finalization work, after the test suite had reached an operational state. It is counted toward Corrective Maintenance and is not double-counted as adaptive or perfective.

Related certificate-generation defect not counted here: the certificate serial-number issue fixed in Martin's work is retained in the existing contribution record and is not reclassified or attributed to Prodip in this section.

File / area	Change and rationale
`tests/run-test-suite.js` and `package.json`	Added explicit discovery, sorting, and passing of matching `tests/test-*.js` files to Taper, avoiding reliance on Windows shell wildcard expansion.
`tests/test-unix.js`	Uses a Windows named pipe on Windows, while retaining the Unix-domain socket path and URL on other platforms.
`tests/browser/start.js`	Starts Karma using the Windows command wrapper on Windows and the existing executable path on other platforms.

File	Change
`.gitattributes`	Enforces LF line endings for `*.sh` files.
`release.sh`; `tests/ssl/gen-test.sh`	Normalized to LF.
`tests/ssl/ca/gen-all-certs.sh` and `gen-server.sh`	Normalized to LF.
`tests/ssl/ca/gen-client.sh` and `gen-localhost.sh`	Normalized to LF.
`tests/ssl/ca/README.md`	Documents the LF requirement for running scripts with Bash/sh.
`tests/test-shell-scripts.js`	Checks six scripts for CR characters and runs `bash -n` on each: 12 checks total.

5. Validation Evidence

5.1 Browser TLS validation

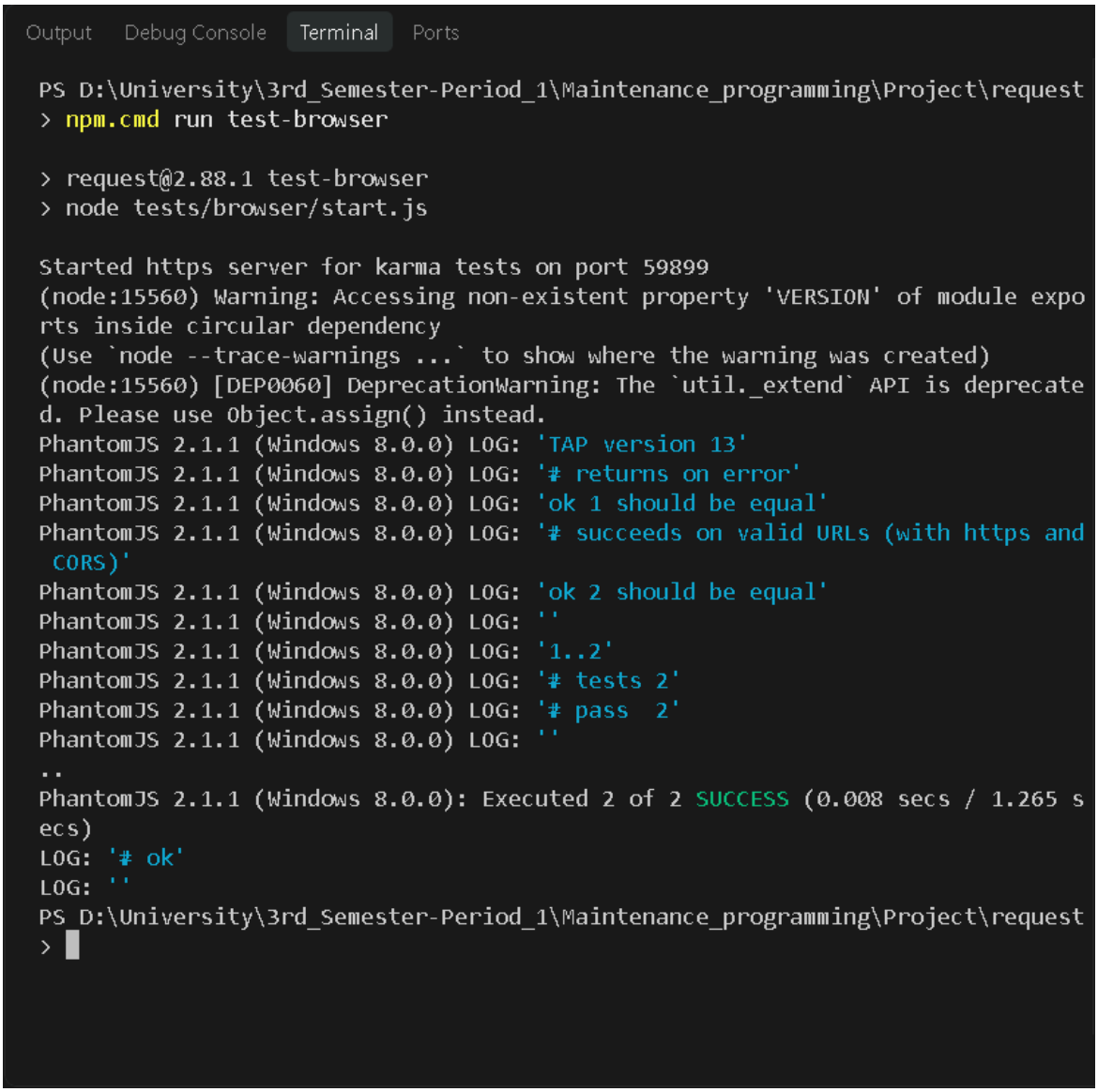


Figure 1. Browser TLS validation with a trusted CA certificate and the correct hostname.

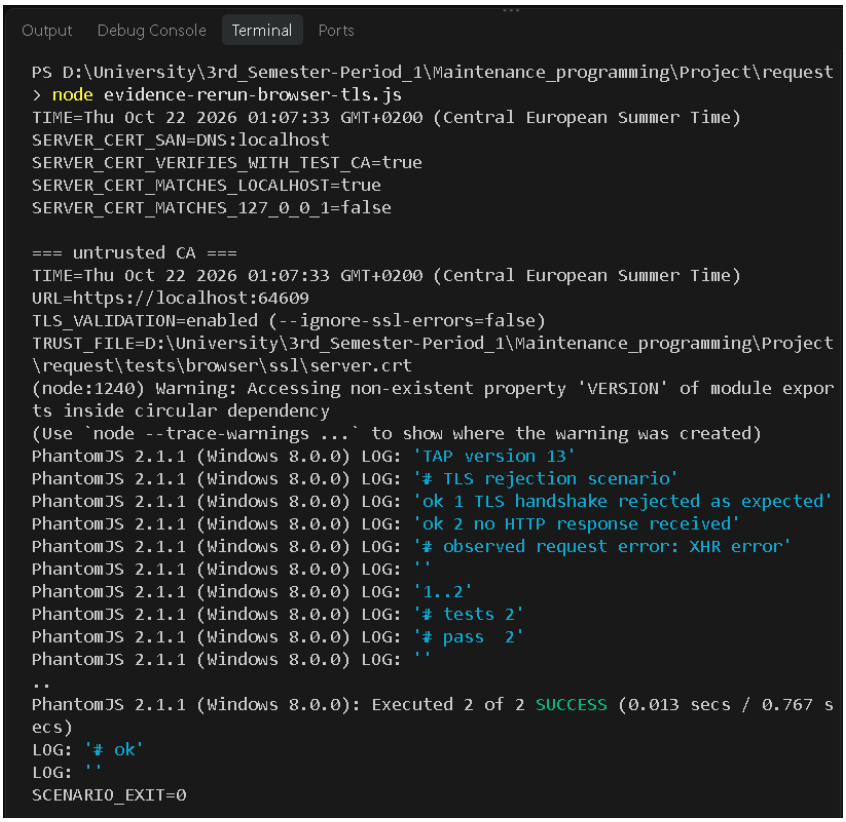


Figure 2. Browser TLS rejection when an untrusted CA certificate is used

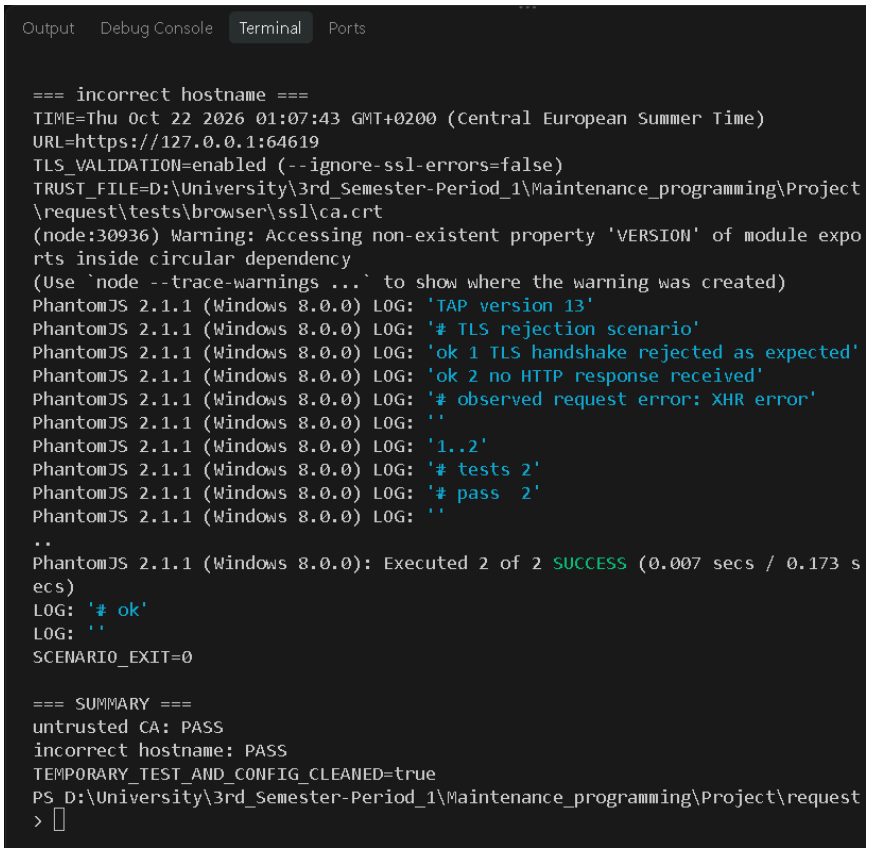


Figure 3. Browser TLS rejection when the hostname does not match the certificate

5.2 Shell-script portability

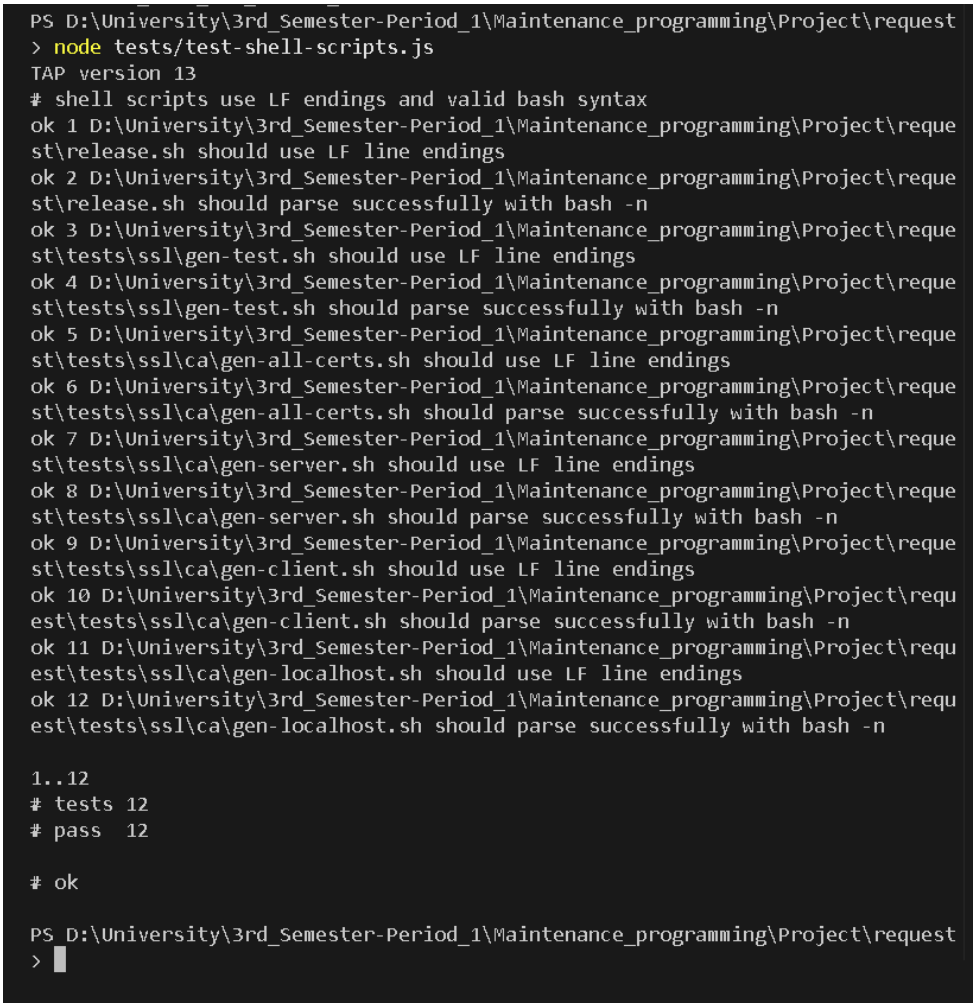


Figure 4. Shell-script portability validation: LF line endings and Bash syntax checks

6. Tests Implemented and Validation

The results below are based on the documented Windows validation of an isolated copy of the working tree. The isolated copy was used to protect the main repository and ignored artifacts during the test run. This document-generation process did not rerun the project tests.

Command / check	Recorded result
`npm.cmd test`	Exit 0; lint passed, 56 matching test files ran, 1,606/1,606 assertions passed, including 2/2 browser assertions.
`npm.cmd run test-cov`	Exit 0; 1,606/1,606 assertions passed.
`npm.cmd run test-browser`	Exit 0; 2/2 browser assertions passed with strict certificate validation enabled.
`node tests/test-https.js`	20/20 passed.
`node tests/test-tunnel.js`	27/27 passed.
`node tests/test-oauth.js`	45/45 passed.
`node tests/test-localAddress.js`	14/14 passed in earlier documented validation.

`node tests/test-unix.js`	6/6 passed in later documented validation; an earlier progress-report snapshot recorded 7/7.
`node tests/test-form-data.js`	64/64 passed in earlier documented validation.
`node tests/test-shell-scripts.js`	12/12 checks passed.
OpenSSL certificate validation	Browser certificate chain and `localhost` hostname verified; certificate was current, 2048-bit RSA/SHA-256, and matched the server key.
Negative browser TLS checks	Browser tests exited nonzero as expected for an unrelated CA and for a trusted-CA certificate with the wrong hostname.
Generator README command	`cd tests/ssl && sh gen-test.sh` was tested in a disposable directory; output files were created under `tests/ssl`, not the parent directory.
`git diff --check`	Passed.

6.1 Environment and limitations

Recorded validation environment: Windows 11 Home (build 26200), Node.js 22.19.0, npm 10.9.3, Windows OpenSSL 3.6.1, WSL Bash 5.2.21 / OpenSSL 3.0.13, and PhantomJS 2.1.1. A direct Windows OpenSSL generator invocation initially failed because its configured default OpenSSL configuration path was unavailable; the documented generator command was validated in WSL with a temporary OpenSSL configuration. No checked-in generator or certificate was regenerated during that README check.

macOS validation. The final branch (`prodip-integrate-andreas`, commit `96ee9b4`) was validated on macOS 15.6.1 with Node.js 22.21.1 and OpenSSL 3.6.2 (Figure 5). Lint passed, all 56 test files passed with 1,606/1,606 assertions, and the browser tests passed 2/2 in PhantomJS 2.1.1 with strict certificate validation enabled. This included `test-https.js` (20/20), `test-tunnel.js` (27/27), `test-oauth.js` (45/45), `test-form-data.js` (64/64) and `test-shell-scripts.js` (12/12). `test-unix.js` uses a Unix-domain socket on macOS and passed 6/6. The coverage run (`npm run test-cov`) gave the same 1,606/1,606 result. The full test output is included in Appendix C (Figure 6).

```
macOS 15.6.1 | Node v22.21.1 | OpenSSL 3.6.2 | commit 96ee9b4 Fix browser TLS test certificate validation
Lint: passed
Test files passed: 56 of 56
ok test-form-data.js ............... 64/64
ok test-https.js ................... 20/20
ok test-oauth.js ................... 45/45
ok test-shell-scripts.js ........... 12/12
ok test-tunnel.js .................. 27/27
ok test-unix.js ..................... 6/6
total ........................ 1606/1606
PhantomJS 2.1.1 (Mac OS X 0.0.0): Executed 2 of 2 SUCCESS (0.003 secs / 0.82 secs)
```

Figure 5: Summary of the full test run on the final branch on macOS: lint, all 56 test files (1,606/1,606 assertions) and the browser tests (2/2) passed.

At the Progress Report 2 checkpoint, only 50 of 55 test files passed on macOS: five tests using the HTTPS test server crashed with `ERR_SSL_EE_KEY_TOO_SMALL` because `tests/ssl/test.key` was a 1024-bit key. Replacing that key (Section 4.3) resolved these failures. The final branch has therefore been validated on both Windows and macOS. It has not been validated on Linux or Node.js 24. PhantomJS remains a legacy browser runtime.

`tests/test-timing.js` fails intermittently on macOS: in 15 repeated runs it failed one assertion 7 times, and the unmodified original code (`master`) failed it at the same rate. The assertion requires the

response of a redirected request to start strictly later than the request itself, measured in whole milliseconds, which does not always hold on a fast machine. This is a pre-existing timing-sensitive test that our changes did not touch, not a regression.

7. Unmaintainable or Difficult-to-Maintain Areas

- Legacy dependencies: the dependency tree contains deprecated packages and peer-dependency conflicts, making a clean install difficult without using the legacy peer-dependency resolver.
- Low diagnostic value of aggregate test output: the initial test command reported a CI failure without clearly identifying the individual failing test, so direct test execution was needed.
- Certificate lifecycle and compatibility: old or expired fixtures and modern OpenSSL security defaults caused failures that could be mistaken for application defects unless certificates and trust behavior were inspected separately.
- Platform-dependent test infrastructure: shell wildcard handling, process launchers, and socket types differed between Windows and Unix-like environments.
- Legacy browser tooling: PhantomJS/Karma introduces constraints and does not represent a modern browser stack.

We addressed some of these difficulties through explicit test discovery, platform-specific test startup and socket handling, stronger test certificates, strict TLS validation, and automated shell-script checks. Dependency modernization and replacement of PhantomJS were not completed and remain possible future work.

8. Challenges and Breakthroughs

Challenge	Approach / breakthrough
Distinguishing application defects from test-fixture problems	Investigated certificate chains and strict versus relaxed HTTPS results before changing `request.js`; identified an expired CA as the initial cause.
Preserving strict security while restoring tests	Replaced and validated test fixtures rather than permanently disabling certificate validation.
Making tests runnable on Windows	Added explicit test discovery, Windows named-pipe handling, and a Windows-specific Karma startup path.
Avoiding false-positive browser TLS results	Configured a custom test CA, disabled SSL-error ignoring, and demonstrated rejection of an unrelated CA and incorrect hostname.
Maintaining script consistency	Enforced LF line endings and added automated CR-character and Bash syntax checks.

9. Individual Contributions

The following sections summarize our individual contributions based on the project records and documented activities. We distinguish implementation work, investigation, testing, and verification where these activities were performed by different members.

9.1 Prodip Kumar Das

- Investigated the initial HTTPS test failure and identified the expired CA certificate as the cause of the strict-verification failures (`CERT_HAS_EXPIRED`).
- Worked on renewing/updating the HTTPS test certificate fixtures and re-ran the HTTPS tests; the recorded result after this stage was 20/20 passing. This initial investigation and work preceded Martin's later macOS validation.
- Worked on the Windows test infrastructure, including explicit test-file discovery, Windows named-pipe support, and browser test startup handling, and validated the suite on Windows after the changes.
- Implemented the browser-specific TLS fixture and Karma configuration updates: replaced browser-only certificate fixtures, enabled strict SSL validation, configured the test CA, and made TLS request errors explicit in test assertions. Trusted-CA and negative CA/hostname cases were checked in an isolated copy.
- Fixed the proxy agent-pool protocol selection issue in `request.js`, exposed by the tunnel regression test. The old-key counterfactual had four protocol errors (23/27 assertions passed), while the updated tunnel test passed 27/27. This is recorded as a separate Corrective Maintenance activity.
- Updated the form-data test server's 401 response handling to send `Connection: close`, preventing `ECONNRESET` while the client was still transmitting a multipart request body. This was part of the core test-suite restoration and is classified under Adaptive Maintenance, not counted as a separate additional activity.
- Completed the separate Perfective Maintenance work for shell-script portability and maintainability after the main suite had become operational: added the `.gitattributes` LF policy, normalized relevant scripts, documented the LF requirement, and added `tests/test-shell-scripts.js` with 12 checks (CR-character checks and `bash -n`).
- Reviewed and verified Martin's later certificate-related changes on Windows. This was verification work and is not claimed as authorship of Martin's certificate-key and serial-number changes.

9.2 Martin Ek

- Project selection. The choice of request as the legacy system, adaptive maintenance as the planned maintenance type, and the preliminary plan based on the "Cover and Modify" approach and Feathers' methodology.
- MacOS verification of the certificate renewal (PR #2). Repeated the HTTPS tests on macOS after the CA renewal and found that the test server crashed with `ERR_SSL_EE_KEY_TOO_SMALL`. Identified the remaining weak or expired fixtures: the renewed CA still used the original 1024-bit key, `tests/ssl/test.key` was 1024-bit, `tests/ssl/test.crt` was SHA-1-signed and expired, and `tests/ssl/ca/client.crt` had expired. Confirmed the cause with a diagnostic run at a lowered OpenSSL security level. These findings, documented in `MAINTENANCE.md`, expanded the project scope from one expired certificate to the whole test certificate set.
- Certificate fixes (PR #3). Replaced `tests/ssl/test.key` with a 2048-bit key and SHA-256 certificate, added `tests/ssl/gen-test.sh` to regenerate them, kept the old key as `tests/ssl/oauth-rsa.key` for the OAuth tests, and fixed the duplicate certificate serial numbers in the generation scripts.
- Final macOS validation. Ran the full test suite, coverage run and browser tests on the final branch on macOS (Section 6.1).

9.3 Andreas Lund

- Project selection. Researched and screened candidate legacy systems. Defined the selection criteria used in the Initial Report: the system had to be declared legacy by its owners rather than argued as legacy by us, and it had to be possible to clone, build and run its test suite as a baseline. Collected the codebase metrics for request (2,270 commits over 2010–2020, 2,991 lines of library code, 8,457 lines of tests across 55 files).
- Complete test-suite investigation (PR #1). After the CA renewal, ran all 55 test files individually on Windows (Node.js 22.19.0), since test-ci reported nothing useful. Found 52 passing (1,434 assertions) and documented the root causes of the three failures (test-tunnel, test-form-data, test-unix) and of the browser-test startup failure.
- Root cause of the unusable test runner. Explained why test-ci ran no tests on Windows (cmd.exe does not expand the tests/test-*.js wildcard).

9.4 Shared work

We shared responsibility for reviewing maintenance work, agreeing on classifications, and preparing the final report and presentation.

10. Overall Impact on the Legacy Code

The maintenance work improved the project's ability to run and validate tests in the documented Windows environment. The test runner now discovers test files explicitly rather than depending on shell wildcard expansion. Platform-specific socket and browser-launch behavior is handled deliberately. Certificate fixtures have been updated, and browser tests now exercise strict certificate validation rather than silently ignoring SSL errors. Shell-script line endings are more consistent, and automated checks help detect line-ending and syntax problems earlier.

Our work also made the investigation more evidence-driven: we distinguished expired or weak test fixtures from potential application defects, retained strict TLS verification, and added regression coverage for tunnel agent-pool behavior. The documented Windows test suite and browser tests passed on the validated snapshot.

The impact has limits. The current branch has not been verified on Linux, the project still uses legacy dependencies and PhantomJS, and the changes do not constitute a complete modernization of the library or its test infrastructure. The maintenance classifications used in this report are summarized in Section 11.

11. Maintenance Classification Summary

Task	Current report treatment	Rationale
HTTPS certificate fixtures	Adaptive maintenance	Updated certificate fixtures and generation to restore HTTPS test compatibility in the current environment.
Windows test discovery, named-pipe support and browser startup	Adaptive maintenance	Adapted the test infrastructure to operate correctly in the target Windows environment.

Form-data test-server response	Adaptive maintenance	Adjusted the test-server response to restore the multipart authentication test as part of core test-suite restoration.
`localAddress` test-only reliability change	Adaptive Maintenance	Replaced reliance on an external endpoint with a local test server to improve test reliability.
Browser TLS configuration and certificate validation	Adaptive Maintenance	Configured browser tests to use the intended CA certificate and hostname validation while retaining strict TLS verification.
Shell-script portability and automated checks	Perfective maintenance	Improved shell-script portability and maintainability through consistent line endings and automated syntax checks.
Agent-pool protocol collision	Corrective Maintenance	Fixed a protocol-selection defect affecting agent pooling; documented as a separate follow-on correction.

We count each activity toward only one primary maintenance type. The HTTPS certificate fixtures, Windows test-infrastructure support, form-data test-server response handling, `localAddress` test isolation, and browser TLS configuration are classified as Adaptive Maintenance. Shell-script portability is classified as Perfective Maintenance. The agent-pool protocol collision is classified as a separate Corrective Maintenance activity.

12. Conclusion

We investigated a legacy Node.js library whose initial test failures involved outdated test fixtures and platform-specific test infrastructure. The maintenance work restored test-suite execution through targeted changes to test discovery, Windows support, certificate fixtures, form-data test-server handling, browser TLS validation, and test isolation. We also improved shell-script portability and corrected a separate agent-pool protocol-selection defect. The final classification treats the core test-suite restoration changes as Adaptive Maintenance, shell-script portability as Perfective Maintenance, and the agent-pool protocol collision as Corrective Maintenance. Recorded Windows and macOS validation passed the documented full test suite and browser checks. The branch has not been validated on Linux, and the project continues to use legacy dependencies and PhantomJS.

Appendix A. Supporting Evidence and Validation Results

The following evidence supports the maintenance activities and validation results documented in this report.

A.1. Browser TLS Validation

- Browser test success with a trusted CA certificate and the correct hostname.
- Expected TLS rejection when an untrusted CA certificate is used.
- Expected TLS rejection when the hostname does not match the certificate.

A.2. Shell-Script Portability

- Automated validation of shell-script line endings and Bash syntax, with all 12 checks passing.

A.3. Test Execution Results

- Windows test-suite validation: 56 test files and 1,606/1,606 assertions passed.
- Browser TLS tests: 2/2 assertions passed with strict TLS validation enabled.
- Shell-script portability checks: 12/12 checks passed.
- macOS test-suite validation: 56 test files and 1,606/1,606 assertions passed, including 2/2 browser assertions.

A.4. Contribution and Classification Records

- The individual contributions are summarized in Section 9. Repository, branch, pull-request, and maintenance-record references are provided in Appendix B.
- Maintenance classifications presented in Section 11.

Appendix B. Project Sources and Supporting Records

The following sources document the project repository, group contributions, progress, maintenance activities, and validation work included in this report.

B.1. Repository and Final Group Branch

- **Source repository:** https://github.com/prodip5619/request
- **Final group branch:** https://github.com/prodip5619/request/tree/prodip-integrate-andreas
- **Final documented commit:** `96ee9b4`

B.2. Pull Requests and Contribution Records

- **Andreas Lund, PR #1 — Document complete test-suite results after certificate fix:** https://github.com/prodip5619/request/pull/1. The changes were merged manually in commit `aa45546`.
- **Martin Ek, PR #2 — Document HTTPS verification on macOS:** https://github.com/prodip5619/request/pull/2
- **Martin Ek, PR #3 — Fix `test.key` and certificate serial numbers:** https://github.com/prodip5619/request/pull/3

B.3. Progress Reports and Maintenance Record

- **Group 4, Progress Report 1:** 27 September 2026.
- **Group 4, Progress Report 2:** 12 October 2026.

- **Project maintenance record:** `MAINTENANCE.md`.

The maintenance record documents the activities and classifications used in the final report. The classifications presented in Section 11 are consistent with the maintenance activities described in Section 4.

Appendix C. Full macOS Test Output

Complete output of `npm run test-ci` on the final branch on macOS, corresponding to the macOS validation summary in Section 6.1.

```
> request@2.88.1 test-ci
> node tests/run-test-suite.js

ok test-agent.js ................. 20/20
ok test-agentOptions.js ............ 8/8
ok test-api.js ..................... 2/2
ok test-aws.js ..................... 15/15
ok test-baseUrl.js ................ 63/63
ok test-basic-auth.js ............. 39/39
ok test-bearer-auth.js ............ 23/23
ok test-body.js ................... 20/20
ok test-cookies.js ................ 25/25
ok test-defaults.js ............... 56/56
ok test-digest-auth.js ............ 15/15
ok test-emptyBody.js ............... 9/9
ok test-errors.js ................. 11/11
ok test-event-forwarding.js ......... 4/4
ok test-follow-all-303.js ........... 3/3
ok test-follow-all.js ............... 3/3
ok test-form-data-error.js .......... 3/3
ok test-form-data.js .............. 64/64
ok test-form-urlencoded.js ........ 28/28
ok test-form.js ................... 19/19
ok test-gzip.js ................... 38/38
ok test-har.js .................... 24/24
ok test-hawk.js ................... 30/30
ok test-headers.js ................ 38/38
ok test-http-signature.js ........... 4/4
ok test-httpModule.js ............. 50/50
ok test-https.js .................. 20/20
ok test-isUrl.js .................. 17/17
ok test-json-request.js ........... 37/37
ok test-localAddress.js ........... 14/14
ok test-multipart-encoding.js ..... 70/70
ok test-multipart.js ............ 172/172
ok test-node-debug.js ............. 20/20
ok test-oauth.js .................. 45/45
ok test-onelineproxy.js ............ 3/3
ok test-option-reuse.js ........... 10/10
ok test-options-convenience-method.js  6/6
ok test-params.js ................. 12/12
ok test-piped-redirect.js ........... 2/2
ok test-pipes.js .................. 36/36
ok test-pool.js ................... 16/16
ok test-promise.js ................. 3/3
ok test-proxy-connect.js ........... 5/5
ok test-proxy.js ............... 104/104
ok test-qs.js ..................... 24/24
ok test-redirect-auth.js .......... 18/18
ok test-redirect-complex.js ....... 40/40
ok test-redirect.js ............... 92/92
ok test-rfc3986.js ................ 44/44
ok test-shell-scripts.js .......... 12/12
ok test-stream.js ................... 2/2
ok test-timeout.js ................ 29/29
ok test-timing.js ................. 43/43
ok test-toJSON.js ................... 7/7
ok test-tunnel.js ................. 27/27
ok test-unix.js ..................... 6/6
total ........................ 1606/1606

ok
```

Figure 6: Full output of the test suite on macOS 15.6.1 (Node.js 22.21.1, OpenSSL 3.6.2): 56/56 test files and 1,606/1,606 assertions passed.

Use of generative AI tools

As permitted by the course, we documented our use of generative AI tools during this phase. We used Claude Code and GitHub Copilot to help run and analyse tests, investigate failures, propose changes, and draft documentation in `MAINTENANCE.md`.

- **Martin:** Verified the certificate fixes on macOS, investigated the remaining HTTPS failures, implemented certificate changes, drafted documentation, and contributed to GitHub-based version control.
- **Andreas:** Helped run individual test files, analysed failures and certificate properties, set up the Node.js 22 environment, drafted documentation, and contributed to GitHub-based version control.
- **Prodip:** Used GitHub Copilot while investigating and implementing certificate and test-infrastructure changes, and contributed to project documentation.

The main benefit of using these tools was increased speed. Running all 56 test files and inspecting the certificates took less time with AI assistance, and the process also helped us identify dependencies that we might otherwise have missed, such as `tests/test-oauth.js` relying on the old test key.

The main challenge was that it sometimes stated incorrect conclusions with confidence, which for example happened when the renewed CA was first described as correct, before its 1024-bit key was found.

Because AI tools can present incorrect information as fact, we manually checked and reproduced key results and reviewed changes through pull requests before accepting them.